

The PyTorch Tensor Cheat-Sheet

CREATE

`torch.tensor([1,2,3])` → from a Python list
`torch.zeros(2,3)` → 2×3 of zeros
`torch.ones(2,3)` → 2×3 of ones
`torch.arange(0,10,2)` → [0,2,4,6,8]
`torch.linspace(0,1,5)` → 5 evenly spaced, 0→1
`torch.rand(2,3)` → uniform [0,1)
`torch.randn(2,3)` → normal (mean 0, std 1)
`torch.eye(3)` → 3×3 identity
`x.clone()` → independent copy

INSPECT

`x.shape` → size of each dimension
`x.dtype` → data type (float32, int64, ...)
`x.device` → cpu or cuda
`x.ndim` → number of dimensions
`x.numel()` → total number of elements

CHANGE TYPE / DEVICE

`x.float()` `x.long()` `x.to(torch.float32)`
`x.to("cuda")` → move to GPU
`x.to("cpu")` → move back

INDEX & SLICE (start:stop:step, 0-based)

`x[0]` → first row
`x[-1]` → last row
`x[:, 0]` → first column (all rows)
`x[1:3]` → rows 1 and 2
`x[x > 0]` → boolean mask: keep positives
`x[[0,2]]` → fancy index: rows 0 and 2

RESHAPE (total stays same)

`x.reshape(3,4)` → new shape, copies if needed
`x.view(3,4)` → same, needs contiguous mem
`x.flatten()` → collapse to 1-D
`x.reshape(-1, 4)` → -1 means "infer this dim"
`x.contiguous()` → fix "view failed" errors

ADD / REMOVE DIMS (size-1 axes)

`x.unsqueeze(0)` (3,) → (1,3) add an axis
`x.squeeze()` (1,3,1) → (3,) drop size-1
`x[None, :]` same as `unsqueeze(0)`

REORDER AXES

`x.permute(2,0,1)` (H,W,C) → (C,H,W)
`x.transpose(0,1)` swap two axes
`x.T` transpose a 2-D tensor

JOIN & SPLIT

`torch.cat([a,b], dim=0)` stack existing axis
`torch.stack([a,b], dim=0)` add a NEW axis
`torch.chunk(x, 3, dim=0)` split into 3
`torch.split(x, 2, dim=0)` pieces of size 2

MATH & REDUCTIONS (dim = axis to collapse)

`x.sum(dim=0)` `x.mean(dim=1)`
`x.max(dim=1)` `x.argmax(dim=1)`
`x + y` `x * y` → element-wise
`a @ b` → matrix multiply
`keepdim=True` → keep reduced axis as size 1

BROADCASTING — THE RULE

Align shapes from the RIGHT. Two dims match when they're equal, or one is 1 (it stretches).

(3,1) + (1,4) → (3,4) ✓ each side stretches
(256,) + (10,256) → (10,256) ✓ trailing dims match
(3,) + (4,) → ERROR ✗ 3 vs 4, neither is 1

Shape error? Line the shapes up on the right and find the axis where they disagree.

Keep this next to your keyboard.

The full book builds every one of these from the ground up — so you know not just what each does, but why.

Get the book: leanpub.com/pytorchfromgroundup